

Практическая работа №8. Бинарные деревья

1. Постройте шаблон дерева поиска. Используйте его для хранения объектов класса С по ключам К в соответствии с таблицей (8.1) (ключи считаются уникальными).
2. Постройте функции добавления и поиска. Создайте интерфейс для функции поиска в виде перегрузки операции [].
3. Введите функцию удаления узла Remove(), которая принимает указатель на удаляемый узел.
4. Унаследуйте класс АВЛ-дерево от класса Дерево поиска. Переопределите функции добавления и удаления узла.

Код 8.1. Класс бинарного дерева поиска

```
#include <iostream>

using namespace std;
//узел
template<class T>
class Node
{
protected:
    //закрытые переменные Node N; N.data = 10 вызовет
ошибку
    T data;

    //не можем хранить Node, но имеем право хранить
указатель
    Node* left;
    Node* right;
    Node* parent;

    //переменная, необходимая для поддержания баланса
дерева
    int height;
public:
    //доступные извне переменные и функции
    virtual void setData(T d) { data = d; }
    virtual T getData() { return data; }
    int getHeight() { return height; }
```

```

virtual Node* getLeft() { return left; }
virtual Node* getRight() { return right; }
virtual Node* getParent() { return parent; }
virtual void setLeft(Node* N) { left = N; }
virtual void setRight(Node* N) { right = N; }
virtual void setParent(Node* N) { parent = N; }

//Конструктор. Устанавливаем стартовые значения для
указателей
Node<T>(T n)
{
    data = n;
    left = right = parent = NULL;
    height = 1;
}

Node<T>()
{
    left = NULL;
    right = NULL;
    parent = NULL;
    data = 0;
    height = 1;
}
virtual void print()
{
    cout << "\n" << data;
}

virtual void setHeight(int h)
{
    height = h;
}

template<class T> friend ostream& operator<<
(ostream& stream, Node<T>& N);};
template<class T>

```

```

    ostream& operator<< (ostream& stream, Node<T>& N)
    {
stream << "\nNode data: " << N.data << ", height: " <<
N.height;
        return stream;
    }
    template<class T>
    void print(Node<T>* N) { cout << "\n" << N-
>getData(); }
    template<class T>
    class Tree
    {
protected:
        //корень - его достаточно для хранения всего дерева
        Node<T>* root;
public:
        //доступ к корневому элементу
        virtual Node<T>* getRoot() { return root; }

        //конструктор дерева: в момент создания дерева ни
одного узла нет, корень смотрит в никуда
        Tree<T>() { root = NULL; }
        //рекуррентная функция добавления узла. Устроена
аналогично, но вызывает сама себя - добавление в левое
или правое поддереву
        virtual Node<T>* Add_R(Node<T>* N)
        {
            return Add_R(N, root);
        }

        virtual Node<T>* Add_R(Node<T>* N, Node<T>*
Current)
        {
            if (N == NULL) return NULL;
            if (root == NULL)
            {
                root = N;
                return N;
            }
        }
    }

```

```

    }
    if (Current->getData() > N->getData())
    {
        //идем влево
        if (Current->getLeft() != NULL)
            Current->setLeft(Add_R(N, Current->getLeft()));
        else
            Current->setLeft(N);
        Current->getLeft()->setParent(Current);
    }

    if (Current->getData() < N->getData())
    {
        //идем вправо
        if (Current->getRight() != NULL)
            Current->setRight(Add_R(N, Current->getRight()));
        else
            Current->setRight(N);
        Current->getRight()->setParent(Current);
    }

    if (Current->getData() == N->getData())
        //нашли совпадение
        ;
    //для несбалансированного дерева поиска
    return Current;
}

//функция для добавления числа. Делаем новый узел с
этими данными и вызываем нужную функцию добавления в
дерево
virtual void Add(T n)
{
    Node<T>* N = new Node<T>;
    N->setData(n);
    Add_R(N);
}

virtual Node<T>* Min(Node<T>* Current=NULL)

```

```

{
    //минимум - это самый "левый" узел. Идём по
    дереву всегда влево
    if (root == NULL) return NULL;
    if(Current==NULL)
        Current = root;
    while (Current->getLeft() != NULL)
        Current = Current->getLeft();

    return Current;
}

virtual Node<T>* Max(Node<T>* Current = NULL)
{
    //минимум - это самый "правый" узел. Идём по
    дереву всегда вправо
    if (root == NULL) return NULL;
    if (Current == NULL)
        Current = root;
    while (Current->getRight() != NULL)
        Current = Current->getRight();

    return Current;
}

//поиск узла в дереве. Второй параметр - в каком
поддереве искать, первый - что искать
virtual Node<T>* Find(int data, Node<T>* Current)
{
    //база рекурсии
    if (Current == NULL) return NULL;
    if (Current->getData() == data) return Current;
    //рекурсивный вызов

    if (Current->getData() > data) return
Find(data, Current->getLeft());
    if (Current->getData() < data) return Find(data,
Current->getRight());
}

```

```

    }
    //три обхода дерева
    virtual void PreOrder(Node<T>* N, void
(*f) (Node<T>*))
    {
        if (N != NULL)
            f(N);
        if (N != NULL && N->getLeft() != NULL)
            PreOrder(N->getLeft(), f);
        if (N != NULL && N->getRight() != NULL)
            PreOrder(N->getRight(), f);
    }
    //InOrder-обход даст отсортированную
последовательность
    virtual void InOrder(Node<T>* N, void
(*f) (Node<T>*))
    {
        if (N != NULL && N->getLeft() != NULL)
            InOrder(N->getLeft(), f);
        if (N != NULL)
            f(N);
        if (N != NULL && N->getRight() != NULL)
            InOrder(N->getRight(), f);
    }

    virtual void PostOrder(Node<T>* N, void
(*f) (Node<T>*))
    {
        if (N != NULL && N->getLeft() != NULL)
            PostOrder(N->getLeft(), f);
        if (N != NULL && N->getRight() != NULL)
            PostOrder(N->getRight(), f);
        if (N != NULL)
            f(N);
    }
};

int main()
{
    Tree<double> T;

```

```

    int arr[15];
    int i = 0;
    for (i = 0; i < 15; i++) arr[i] = (int)(100 *
cos(15 * double(i+1)));
    for (i = 0; i < 15; i++)
        T.Add(arr[i]);
    Node<double>* M = T.Min();
    cout << "\nMin = " << M->getData() << "\tFind " <<
arr[3] << ": " << T.Find(arr[3], T.getRoot());

    void (*f_ptr)(Node<double>*); f_ptr = print;
    cout << "\n-----\nInorder:";
    T.InOrder(T.getRoot(), f_ptr);
    char c; cin >> c;
    return 0;
}

```

Таблица 7.1. Ключ и тип объекта, хранимого в классе АВЛ-дерево

Вариант	Ключ	Класс С
1.	Адрес	«Объект жилой недвижимости». Минимальный набор полей: адрес, тип (перечислимый тип: городской дом, загородный дом, квартира, дача), общая площадь, жилая площадь, цена.
2.	Название	«Сериал». Минимальный набор полей: название, продюсер, количество сезонов, популярность, рейтинг, дата запуска, страна.
3.	Название	«Смартфон». Минимальный набор полей: название, размер экрана, количество камер, объем аккумулятора, максимальное количество часов без подзарядки, цена.
4.	Фамилия и имя	«Спортсмен». Минимальный набор полей: фамилия, имя, возраст, гражданство, вид спорта, количество медалей.
5.	Фамилия и имя	«Врач». Минимальный набор полей: фамилия, имя, специальность,

		должность, стаж, рейтинг (вещественное число от 0 до 100).
6.	Международный код	«Авиакомпания». Минимальный набор полей: название, международный код, количество обслуживаемых линий, страна, интернет-адрес сайта, рейтинг надёжности (целое число от -10 до 10).
7.	Название	«Книга». Минимальный набор полей: фамилия (первого) автора, имя (первого) автора, название, год издания, название издательства, число страниц, вид издания (перечислимый тип: электронное, бумажное или аудио), тираж.
8.	Номер в каталоге	«Небесное тело». Минимальный набор полей: тип (перечислимый тип: астероид, естественный спутник, планета, звезда, квазар), имя (может отсутствовать), номер в небесном каталоге, удаление от Земли, расчётная масса в миллиардах тонн (для сверхбольших объектов допускается значение Inf, которое должно корректно обрабатываться).
9.	Название	«Населённый пункт». Минимальный набор полей: название, тип (перечислимый тип: город, посёлок, село, деревня), числовой код региона, численность населения, площадь.
10.	Имя или псевдоним исполнителя, название альбома	«Музыкальный альбом». Минимальный набор полей: имя или псевдоним исполнителя, название альбома, количество композиций, год выпуска, количество проданных экземпляров.
11.	Серийный номер	«Автомобиль». Минимальный набор полей: имя модели, название производителя, цвет, серийный номер, количество дверей, год выпуска, цена.

12.	Регистр ационн ый номер автомоб иля	«Автовладелец». Минимальный набор полей: фамилия, имя, регистрационный номер автомобиля, дата рождения, номер техпаспорта.
13.	Назван ие фильма	«Фильм». Минимальный набор полей: фамилия, имя режиссёра, название, страна, год выпуска, стоимость, доход.
14.	Назван ие, год построй ки	«Стадион». Минимальный набор полей: название, виды спорта, год постройки, вместимость, количество арен.
15.	Назван ие, город	«Спортивная Команда». Минимальный набор полей: название, город, число побед, поражений, ничьих, количество очков.
16.	Фамили я и имя	«Пациент». Минимальный набор полей: фамилия, имя, дата рождения, телефон, адрес, номер карты, группа крови.
17.	Фамили я и имя	«Покупатель». Минимальный набор полей: фамилия, имя, город, улица, номера дома и квартиры, номер счёта, средняя сумма чека.
18.	Фамили я и имя	«Школьник». Минимальный набор полей: фамилия, имя, пол, класс, дата рождения, адрес.
19.	Назван ие	«Государство». Минимальный набор полей: название, столица, язык, численность населения, площадь.
20.	Адрес	«Сайт». Минимальный набор полей: название, адрес, дата запуска,

		язык, тип (блог, интернет-магазин и т.п.), cms, дата последнего обновления, количество посетителей в сутки.
21.	Фамилия и имя	«Человек». Минимальный набор полей: фамилия, имя, пол, рост, возраст, вес, дата рождения, телефон, адрес.
22.	Название	«Программа». Минимальный набор полей: название, версия, лицензия, есть ли версия для android, iOS, платная ли, стоимость, разработчик, открытость кода, язык кода.
23.	Производитель, модель	«Ноутбук». Минимальный набор полей: производитель, модель, размер экрана, процессор, количество ядер, объем оперативной памяти, объем диска, тип диска, цена.
24.	Марка, диаметр колеса	«Велосипед». Минимальный набор полей: марка, тип, тип тормозов, количество колес, диаметр колеса, наличие амортизаторов, детский или взрослый.
25.	Фамилия и имя	«Программист». Минимальный набор полей: фамилия, имя, email, skype, telegram, основной язык программирования, текущее место работы, уровень (число от 1 до 10).
26.	Псевдоним	«Профиль в соц.сети». Минимальный набор полей: псевдоним, адрес страницы, возраст, количество друзей, интересы, любимая цитата.
27.	Псевдоним	«Супергерой». Минимальный набор полей: псевдоним, настоящее имя, дата рождения, пол, суперсила, слабости, количество побед, рейтинг силы.
28.	Производитель, модель	«Фотоаппарат». Минимальный набор полей: производитель, модель, тип, размер матрицы, количество мегапикселей, вес, тип карты памяти, цена.
29.	Полный адрес	«Файл». Минимальный набор полей: полный адрес, краткое имя, дата последнего изменения, дата последнего чтения, дата создания.
30.	Произв	«Самолет».

	одитель , названи е	Минимальный набор полей: название, производитель, вместимость, дальность полета, максимальная скорость.
--	--	---

Задание 8.2

Используйте шаблон класса Heap (куча, пирамида) для хранения объектов в соответствии с таблицей 8.2 (используется упорядоченность по приоритету, в корне дерева – максимум). Реализуйте функции просеивания элементов вверх SiftUp() и вниз SiftDown() = Heapify(), на их основе реализуйте функции добавления узла push() и удаления корня дерева ExtractMax(). Выведите элементы Heap в порядке убывания приоритета с её помощью. Добавьте операцию удаления произвольного элемента Remove() (требуется просеивание вверх или вниз?) и деструктор.

Таблица 8.2. Варианты типов хранимых в контейнере значений и порядок сравнения полей для упорядочения объектов

Вариант	Класс C	Приоритет
1.	«Объект жилой недвижимости». Минимальный набор полей: адрес, тип (перечислимый тип: городской дом, загородный дом, квартира, дача), общая площадь, жилая площадь, цена.	Цена; адрес (по возрастанию)
2.	«Сериал». Минимальный набор полей: название, продюсер, количество сезонов, популярность, рейтинг, дата запуска, страна.	Рейтинг; название (по возрастанию)
3.	«Смартфон». Минимальный набор полей: название, размер экрана, количество камер, объем аккумулятора, максимальное количество часов без подзарядки, цена.	Цена, количество камер, размер экрана; название марки (по возрастанию)
4.	«Спортсмен». Минимальный набор полей: фамилия, имя,	Количество медалей; возраст (по

	возраст, гражданство, вид спорта, количество медалей.	возрастанию); фамилия и имя (по возрастанию)
5.	«Врач». Минимальный набор полей: фамилия, имя, специальность, должность, стаж, рейтинг (вещественное число от 0 до 100).	Рейтинг, стаж; фамилия и имя (по возрастанию)
6.	«Авиакомпания». Минимальный набор полей: название, международный код, количество обслуживаемых линий, страна, интернет-адрес сайта, рейтинг надёжности (целое число от -10 до 10).	Надёжность, количество обслуживаемых линий; название (по возрастанию)
7.	«Книга». Минимальный набор полей: фамилия (первого) автора, имя (первого) автора, название, год издания, название издательства, число страниц, вид издания (перечислимый тип: электронное, бумажное или аудио), тираж.	Тираж; год издания (по возрастанию); название (по возрастанию)
8.	«Небесное тело». Минимальный набор полей: тип (перечислимый тип: астероид, естественный спутник, планета, звезда, квазар), имя (может отсутствовать), номер в небесном каталоге, удаление от Земли, расчётная масса в миллиардах тонн (для сверхбольших объектов допускается значение Inf, которое должно корректно обрабатываться).	Масса; номер в каталоге (по возрастанию)
9.	«Населённый пункт». Минимальный набор полей: название, тип (перечислимый тип: город, посёлок, село, деревня), числовой код региона, численность населения, площадь.	Площадь, численность населения; числовой код региона (по возрастанию)
10.	«Музыкальный альбом». Минимальный набор полей: имя или	Количество проданных

	псевдоним исполнителя, название альбома, количество композиций, год выпуска, количество проданных экземпляров.	экземпляров; количество композиций; год выпуска (по возрастанию); имя или псевдоним исполнителя (по возрастанию)
11.	«Фильм». Минимальный набор полей: фамилия, имя режиссёра, название, страна, год выпуска, стоимость, доход.	Доход, стоимость; год выпуска (по возрастанию); фамилия и имя режиссера (по возрастанию); название фильма (по возрастанию)
12.	«Автомобиль». Минимальный набор полей: имя модели, цвет, серийный номер, количество дверей, год выпуска, цена.	Цена; год выпуска; марка (по возрастанию); серийный номер (по возрастанию)
13.	«Автовладелец». Минимальный набор полей: фамилия, имя, регистрационный номер автомобиля, дата рождения, номер техпаспорта.	Регистрационный номер автомобиля; номер техпаспорта; фамилия и имя автовладельца (по возрастанию)
14.	«Стадион». Минимальный набор полей: название, виды спорта, год постройки, вместимость, количество арен.	Вместимость, количество арен, год постройки; название (по возрастанию)
15.	«Спортивная Команда». Минимальный набор полей: название, город, число побед, поражений, ничьих, количество очков.	Число побед, число ничьих; число поражений (по возрастанию); название (по

		возрастанию)
16.	«Пациент». Минимальный набор полей: фамилия, имя, дата рождения, телефон, адрес, номер карты, группа крови.	Номер карты; группа крови; фамилия и имя (по возрастанию)
17.	«Покупатель». Минимальный набор полей: фамилия, имя, город, улица, номера дома и квартиры, номер счёта, средняя сумма чека.	Средняя сумма чека; номер счёта; фамилия и имя (по возрастанию)
18.	«Школьник». Минимальный набор полей: фамилия, имя, пол, класс, дата рождения, адрес.	Класс; дата рождения (по возрастанию); фамилия и имя (по возрастанию)
19.	«Человек». Минимальный набор полей: фамилия, имя, пол, рост, возраст, вес, дата рождения, телефон, адрес.	Возраст, рост; вес (по возрастанию); фамилия и имя (по возрастанию)
20.	«Государство». Минимальный набор полей: название, столица, язык, численность населения, площадь.	Численность населения; площадь; название (по возрастанию)
21.	«Сайт». Минимальный набор полей: название, адрес, дата запуска, язык, тип (блог, интернет-магазин и т.п.), cms, дата последнего обновления, количество посетителей в сутки.	Количество посетителей в сутки, дата последнего обновления; адрес (по возрастанию)
22.	«Программа». Минимальный набор полей: название, версия, лицензия, есть ли версия для android, iOS, платная ли, стоимость, разработчик, открытость кода, язык кода.	Стоимость, версия; название (по возрастанию)
23.	«Ноутбук». Минимальный набор полей: производитель, модель, размер экрана, процессор, количество ядер, объем оперативной памяти, объем диска, тип	Цена, количество ядер, объем оперативной памяти, размер экрана; модель (по

	диска, цена.	возрастанию)
24.	«Велосипед». Минимальный набор полей: марка, тип, тип тормозов, количество колес, диаметр колеса, наличие амортизаторов, детский или взрослый.	Диаметр колеса, количество колес; марка (по возрастанию)
25.	«Программист». Минимальный набор полей: фамилия, имя, email, skype, telegram, основной язык программирования, текущее место работы, уровень (число от 1 до 10).	Уровень; основной язык программирования (по возрастанию); фамилия и имя (по возрастанию)
26.	«Профиль в соц.сети». Минимальный набор полей: псевдоним, адрес страницы, возраст, количество друзей, интересы, любимая цитата.	Количество друзей; псевдоним (по возрастанию)
27.	«Супергерой». Минимальный набор полей: псевдоним, настоящее имя, дата рождения, пол, суперсила, слабости, количество побед, рейтинг силы.	Рейтинг силы, количество побед; псевдоним (по возрастанию)
28.	«Фотоаппарат». Минимальный набор полей: производитель, модель, тип, размер матрицы, количество мегапикселей, вес, тип карты памяти, цена.	Цена, вес, размер матрицы; модель (по возрастанию)
29.	«Файл». Минимальный набор полей: полный адрес, краткое имя, дата последнего изменения, дата последнего чтения, дата создания.	Дата последнего изменения, дата последнего чтения; полный адрес (по возрастанию)
30.	«Самолет». Минимальный набор полей: название, производитель, вместимость, дальность полета, максимальная скорость.	Вместимость, дальность полета; производитель (по возрастанию), название (по возрастанию)

Код 7.2. Класс Heap (куча, пирамида)

```
#include <iostream>
using namespace std;

//узел дерева
template <class T>
class Node
{
private:
    T value;
public:
    //установить данные в узле
    T getValue() { return value; }
    void setValue(T v) { value = v; }

    //сравнение узлов
    int operator<(Node N)
    {
        return (value < N.getValue());
    }
    int operator>(Node N)
    {
        return (value > N.getValue());
    }
    //вывод содержимого одного узла
    void print()
    {
        cout << value;
    }
};

template <class T>
void print(Node<T>* N)
{
    cout << N->getValue() << "\n";
}
```



```

//куча (heap)
template <class T>
class Heap
{
private:
//массив
Node<T>* arr;
//сколько элементов добавлено
int len;
//сколько памяти выделено
int size;
public:
//доступ к вспомогательным полям кучи и оператор
индекса
int getCapacity() { return size; }
int getCount() { return len; }

Node<T>& operator[](int index)
{
    if (index < 0 || index >= len)
        ;//?
    return arr[index];}
//конструктор
Heap<T> (int MemorySize = 100)
{
    arr = new Node<T>[MemorySize];
    len = 0;
    size = MemorySize;
}
//поменять местами элементы arr[index1],
arr[index2]
void Swap(int index1, int index2)
{
    if (index1 <= 0 || index1 >= len)
        ;
    if (index2 <= 0 || index2 >= len)
        ;
    //здесь нужна защита от дурака

```

```

        Node<T> temp;
        temp = arr[index1];
        arr[index1] = arr[index2];
arr[index2] = temp;
    }

    //скопировать данные между двумя узлами
void Copy(Node<T>* dest, Node<T>* source)
{
    dest->setValue(source->getValue());
}

    //функции получения левого, правого дочернего
элемента, родителя или их индексов в массиве
Node<T>* GetLeftChild(int index)
{
    if (index < 0 || index * 2 >= len)
        ;

    //здесь нужна защита от дурака
    return &arr[index * 2 + 1];
}
Node<T>* GetRightChild(int index)
{
    if (index < 0 || index * 2 >= len);
    //здесь нужна защита от дурака
    return &arr[index * 2 + 2];
}
Node<T>* GetParent(int index)
{
    if (index <= 0 || index >= len)
        ;

    //здесь нужна защита от дурака

    if (index % 2 == 0)
        return &arr[index / 2 - 1];
    return &arr[index / 2];
}

```

```

int GetLeftChildIndex(int index)
{
    if (index < 0 || index * 2 >= len)
        ;
    //здесь нужна защита от дурака
    return index * 2 + 1;
}

int GetRightChildIndex(int index)
{
    if (index < 0 || index * 2 >= len)
        ;
    //здесь нужна защита от дурака

    return index * 2 + 2;
}

int GetParentIndex(int index)
{
    if (index <= 0 || index >= len)
        ;
    //здесь нужна защита от дурака
    if (index % 2 == 0)
        return index / 2 - 1;
    return index / 2;
}

//просеять элемент вверх
void SiftUp(int index = -1)
{
    if (index == -1) index = len - 1;
    int parent = GetParentIndex(index);
    int index2 = GetLeftChildIndex(parent);
    if (index2 == index) index2 =
GetRightChildIndex(parent);
    int max_index = index;

    if (index < len && index2 < len && parent >= 0)
{

```

```

        if (arr[index] > arr[index2])
            max_index = index;
        if (arr[index] < arr[index2])
            max_index = index2;
    }
    if (parent < len && parent >= 0 &&
arr[max_index]>arr[parent])
    {
        //нужно просеивание вверх
        Swap(parent, max_index);
        SiftUp(parent);
    }
}
//добавление элемента - вставляем его в конец
массива и просеиваем вверх
template <class T>
void push(T v)
{
    Node<T>* N = new Node<T>;
    N->setValue(v);
    push(N);
}
template <class T>
void push(Node<T>* N)
{
    if (len < size)
    {
        Copy(&arr[len], N);
        len++;SiftUp();}}
//перечислить элементы кучи и применить к ним
функцию
void Straight(void(*f)(Node<T>*))
{
    int i;
    for (i = 0; i < len; i++)
    {
        f(&arr[i]);
    }
}

```

```

    }

    //перебор элементов, аналогичный проходам бинарного
    дерева
    void InOrder(void(*f) (Node<T>*), int index = 0)
    {
        if (GetLeftChildIndex(index) < len)
            PreOrder(f, GetLeftChildIndex(index));
        if (index >= 0 && index < len)
            f(&arr[index]);
        if (GetRightChildIndex(index) < len)
            PreOrder(f, GetRightChildIndex(index));
    }
};

int main()
{
    Heap<int> Tree;

    Tree.push(1);
    Tree.push(-1);
    Tree.push(-2);
    Tree.push(2);
    Tree.push(5);
    Tree.push(6);
    Tree.push(-3);
    Tree.push(-4);
    cout << "\n-----\nStraight:";
    void(*f_ptr) (Node<int>*); f_ptr = print;
    Tree.Straight(f_ptr);
    return 0;
}

```